

Sistemi Context aware: Campus Assistant

Nicola Tinari

nicola.tinari@studio.unibo.it

mat: 0001231663

2025/26

Obiettivi e Requisiti

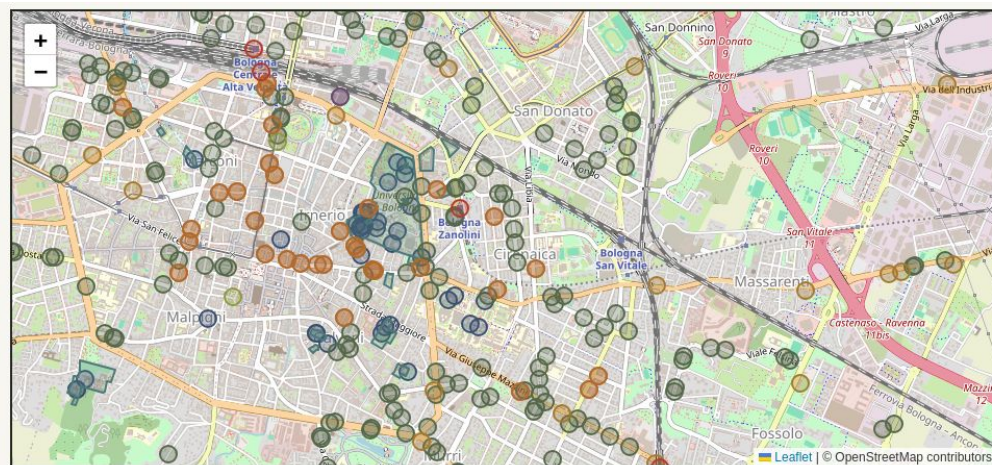
- Realizzare un sistema che supporti lo studente durante la vita universitaria suggerendo servizi e punti di interesse in base al contesto corrente (la posizione, l'orario e le preferenze personali).
- La stessa posizione produce suggerimenti diversi al mattino, in pausa pranzo o a fine giornata
- Il sistema possiede anche un'agenda incorporata che avvisa l'utente di un impegno imminente
- L'app mobile è dotata anche di un sistema offline
- Altri servizi:
 - Geofencing
 - Feedback sui suggerimenti
 - Analytics spaziali e temporali
 - Containerizzazione (Docker)

GEOFENCING_ENTER

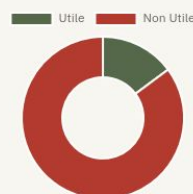
Sei entrato nell' area di Biblioteca del Dipartimento di Filosofia e Comunicazione - FILCOM. Sezione di Filosofia.

Motivo: Geofence scattato - L'utente ha attraversato il confine virtuale del POI: Biblioteca del Dipartimento di Filosofia e Comunicazione - FILCOM. Sezione di Filosofia

Feedback registrato



■ Biblioteca ■ Sala Studio ■ Mensa ■ Ufficio ■ Segreteria ■ Fermata ■ Noleggio bici ■ Stazione ■ Benzinario



SUGGERIMENTO

Sei nei paraggi di Università di Bologna, potrebbe interessarti!

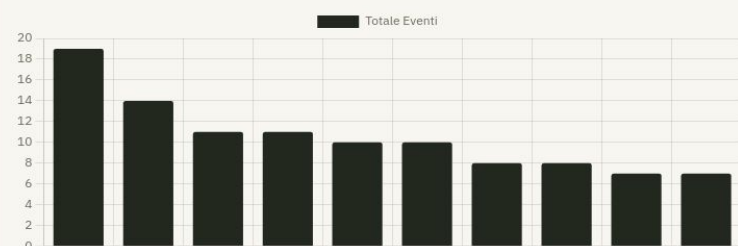
Motivo: Miglior POI per contesto attuale (Punteggio: 2500.0)

Feedback registrato

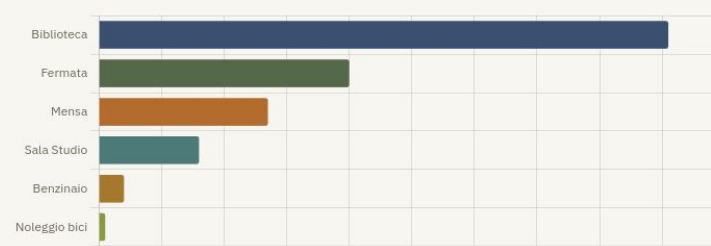


ANALYTICS

POI con più interazioni



Categorie più richieste



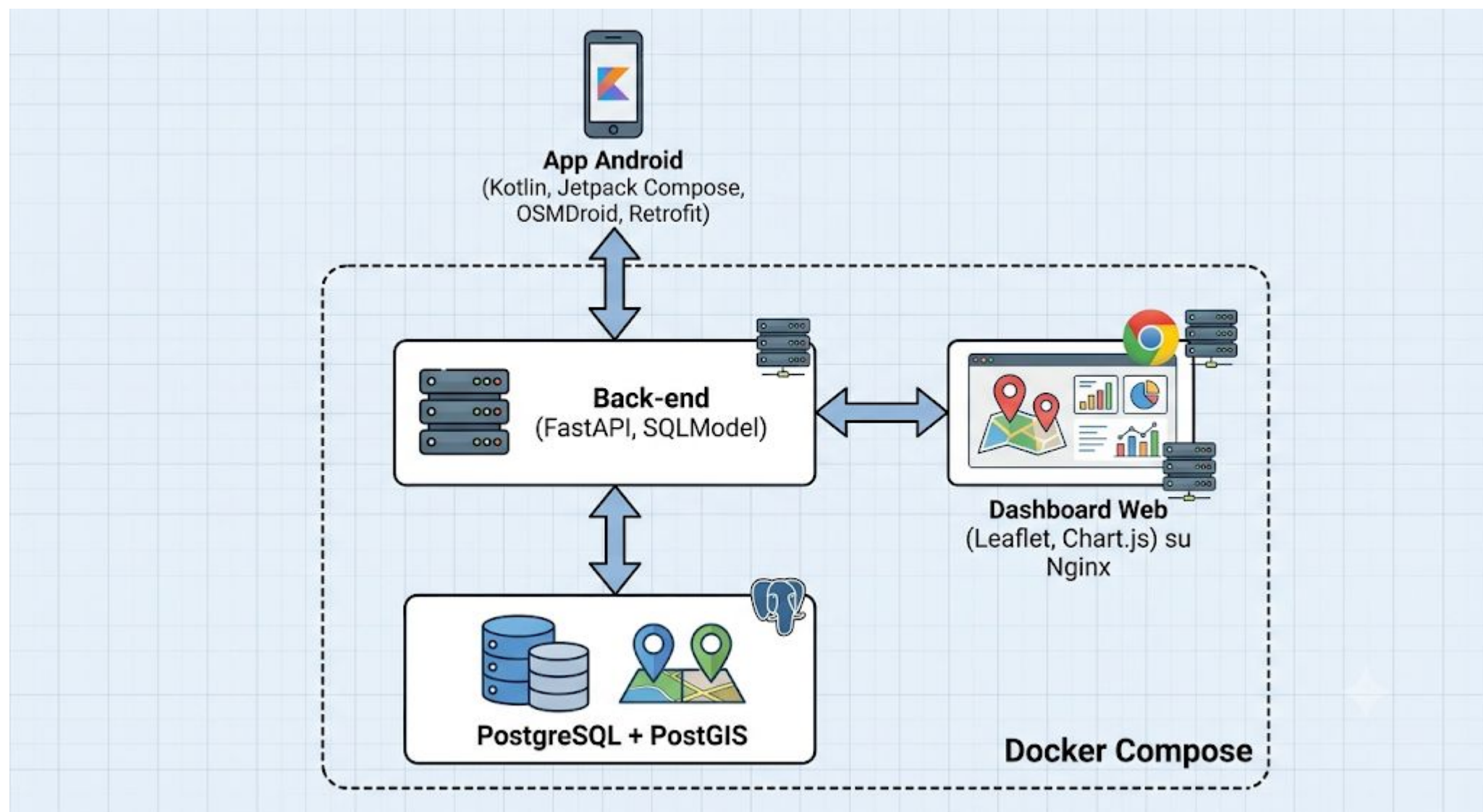
Architettura

Il sistema è organizzato in tre componenti che comunicano esclusivamente tramite API REST:

- un'app Android (Kotlin)
- un back-end (FastAPI)
- una dashboard web amministrativa (HTML/JS)

Tutta la logica contestuale risiede sul server, mentre i client si occupano della presentazione

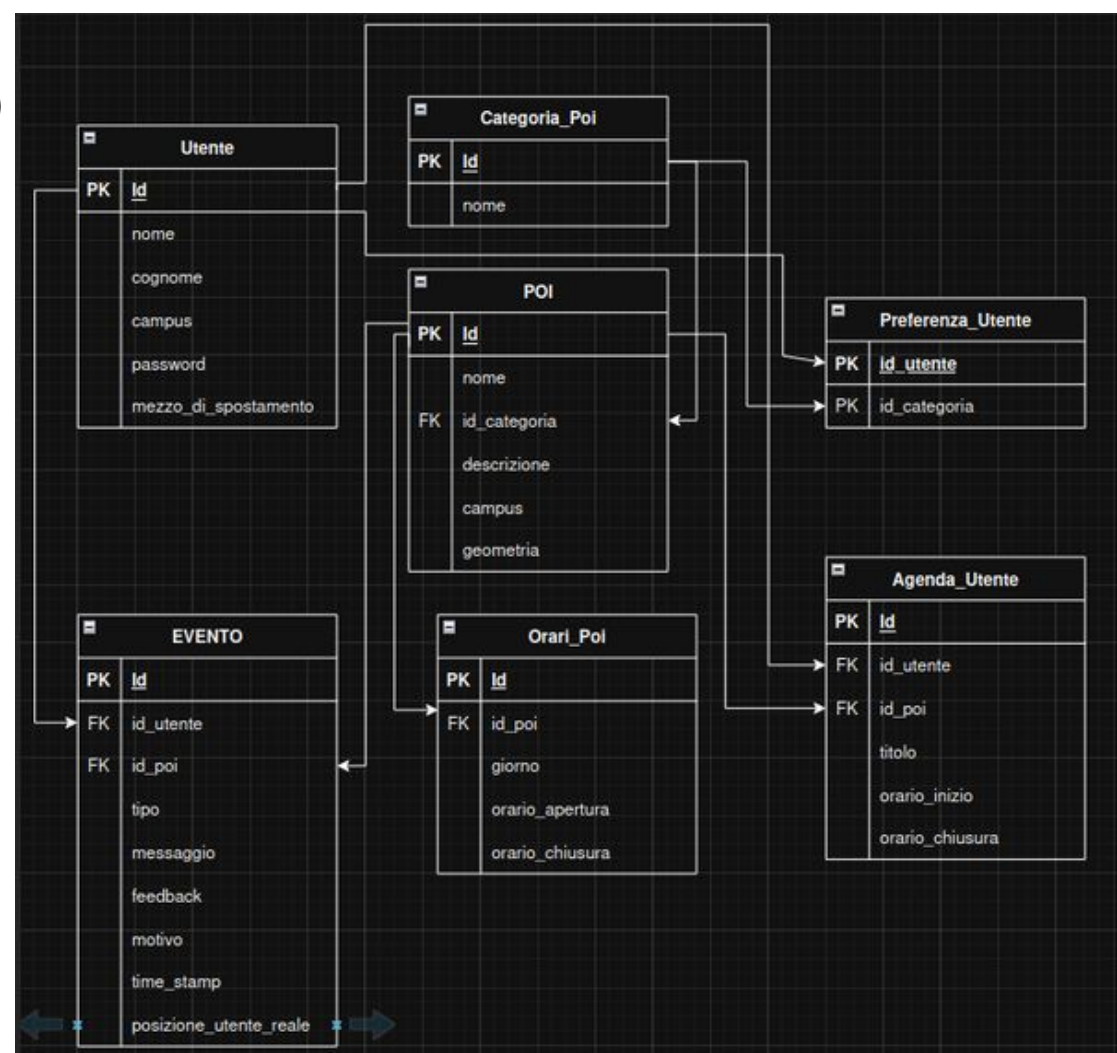
Deploy con Docker Compose





Modello Base di Dati

- L'entità principale del DB sono i punti di interesse (POI), memorizzati in PostgreSQL con estensione PostGIS come geometrie reali, punti o poligoni. Ovviamente ognuno con i propri orari
- Attorno ai Poi ruotano le altre entità:
 - categorie
 - utenti (con il proprio mezzo di spostamento)
 - preferenze dei vari utenti
 - eventi (tracciano ogni tipo di notifica/suggerimento)
 - orari dei poi
 - agenda utente
- Geometrie in EPSG:4326 (compatibili col GPS)
- Calcoli metrici avvengono nel DB riproiettando in EPSG:32632





BackEnd

- Il back-end espone le proprie funzionalità tramite API REST, documentate automaticamente con Swagger (<http://127.0.0.1:8000/docs>)
- Organizzazione dei file per livello e responsabilità:
 - modelli (Public, Base, Create ed eventualmente altri)
 - servizi
 - router
- Ogni file gestisce un'entità del database (CRUD)
- Servizi speciali:
 - geofence_service
 - auth_service
 - analytics_service
 - raccomandation_service

DELETE	/utenti/{utente_id}	Delete Utente	▼
GET	/utenti/campus/{campus}	Get Utenti By Campus	▼
GET	/utenti/mezzo/{mezzo}	Get Utenti By Mezzo	▼
GET	/utenti/preferenza/{id_categoria}	Get Utenti By Preferenza	▼
POI ^			
GET	/poi/	Get All Pois	▼
POST	/poi/	Create Poi	▼
GET	/poi/nearby	Get Pois Nearby	▼
GET	/poi/nearby-with-distance	Get Pois Nearby With Distance	▼
GET	/poi/search	Search Pois	▼
GET	/poi/filter	Get Filtered Pois	▼
GET	/poi/{poi_id}	Get Poi By Id	▼
PATCH	/poi/{poi_id}	Update Poi	▼
DELETE	/poi/{poi_id}	Delete Poi	▼



Regola di ranking

- La rilevanza di un POI è calcolata con una regola esplicita che assegna ad ogni POI nel raggio di 2km un punteggio.
- basata su tre fattori contestuali:
 - distanza
 - orario di apertura
 - preferenze dell'utente (tabella: PREFERENZA_UTENTE)

$$P = \frac{1000}{d} \times K_{pref} \times S_{open}$$

P : Punteggio finale del Poi

d : Distanza dal Poi in metri

K_{pref} : Fattore di preferenza pari a 2,5 se la categoria è tra i preferiti o derivata dal mezzo, altrimenti 1

S_{open} : Fattore di stato pari a 0 se il Poi è chiuso nell'orario corrente, 1 se è aperto

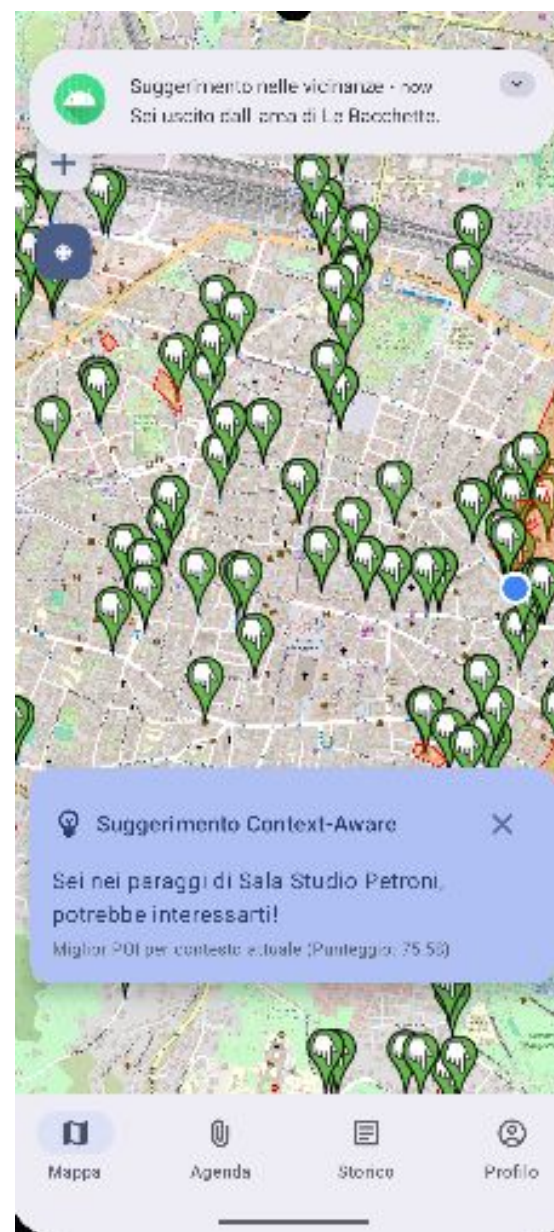
esempio:

Il locale Crock! ha avuto un punteggio 25.25 alle ore 12:23 di un mercoledì, alle 16:00 non è stato suggerito essendo chiuso dalle 15:30



App: Mappa e Poi vicini

- L'app è sviluppata in Kotlin con Jetpack Compose e architettura MVVM.
- La schermata principale mostra su mappa OSMDroid la posizione dell'utente e i Poi
- Ogni marker è selezionabile per visualizzarne i dettagli
- Il pulsante *Servizi vicini* apre la lista dei primi 20 Poi ordinati per rilevanza, calcolata dal back-end sul contesto corrente.
- All'avvio l'app riceve inoltre un suggerimento contestuale mostrato come card sulla mappa.





App Mobile: Profilo, Storico e Feedback

- Nella schermata profilo l'utente configura:
 - le categorie di interesse
 - il mezzo di spostamento, che viene mappato su una categoria rilevante (chi usa l'auto riceve i benzinai, chi usa l'autobus le fermate...)
- Queste scelte influenzano direttamente il ranking
- La schermata dei suggerimenti mostra invece
 - lo storico personale di tutti gli eventi ricevuti
 - il contesto in cui sono stati generati
 - permette di esprimere un feedback *Utile / Non utile*

Informazioni Personali

Nome: Nicola
Cognome: Tinari

Campus di preferenza
Bologna

Mezzo di Spostamento
A_PIEDI

Aggiorna Info

Le tue Preferenze

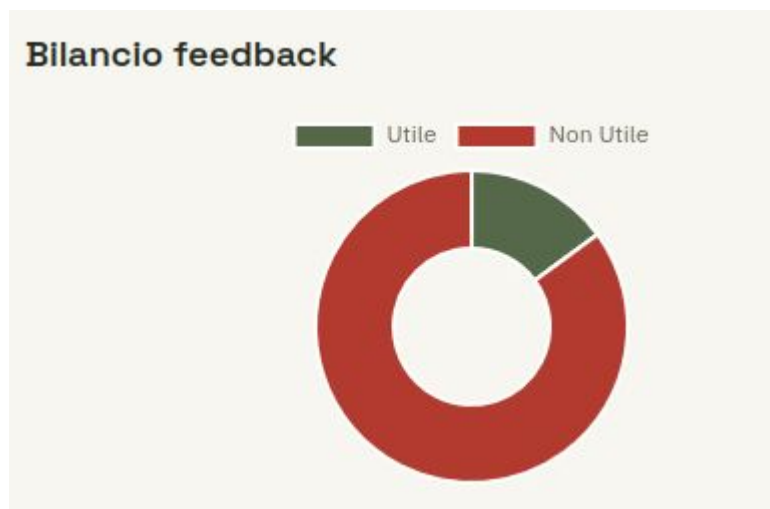
Seleziona le categorie dei servizi che usi più spesso:

Biblioteca Sala studio Mensa

Ufficio Segreteria Fermata

Noleggio bici Stazione

Benzinaio



Storico Notifiche

GEOFENCING_ENTER
Sei entrato nell' area di Sala Studio Petroni.
Motivo: Geofence scattato - L'utente ha attraversato il confine virtuale del POI: Sala Studio Petroni.

Feedback registrato ✓ ✗

SUGGERIMENTO
Sei nei paraggi di Sala Studio Petroni, potrebbe interessarti!
Motivo: Miglior POI per contesto attuale (Punteggio: 75.58)

Feedback registrato ✓ ✗

POI_SELEZIONATO
Hai selezionato il seguente luogo dalla mappa
Motivo: Marker cliccato

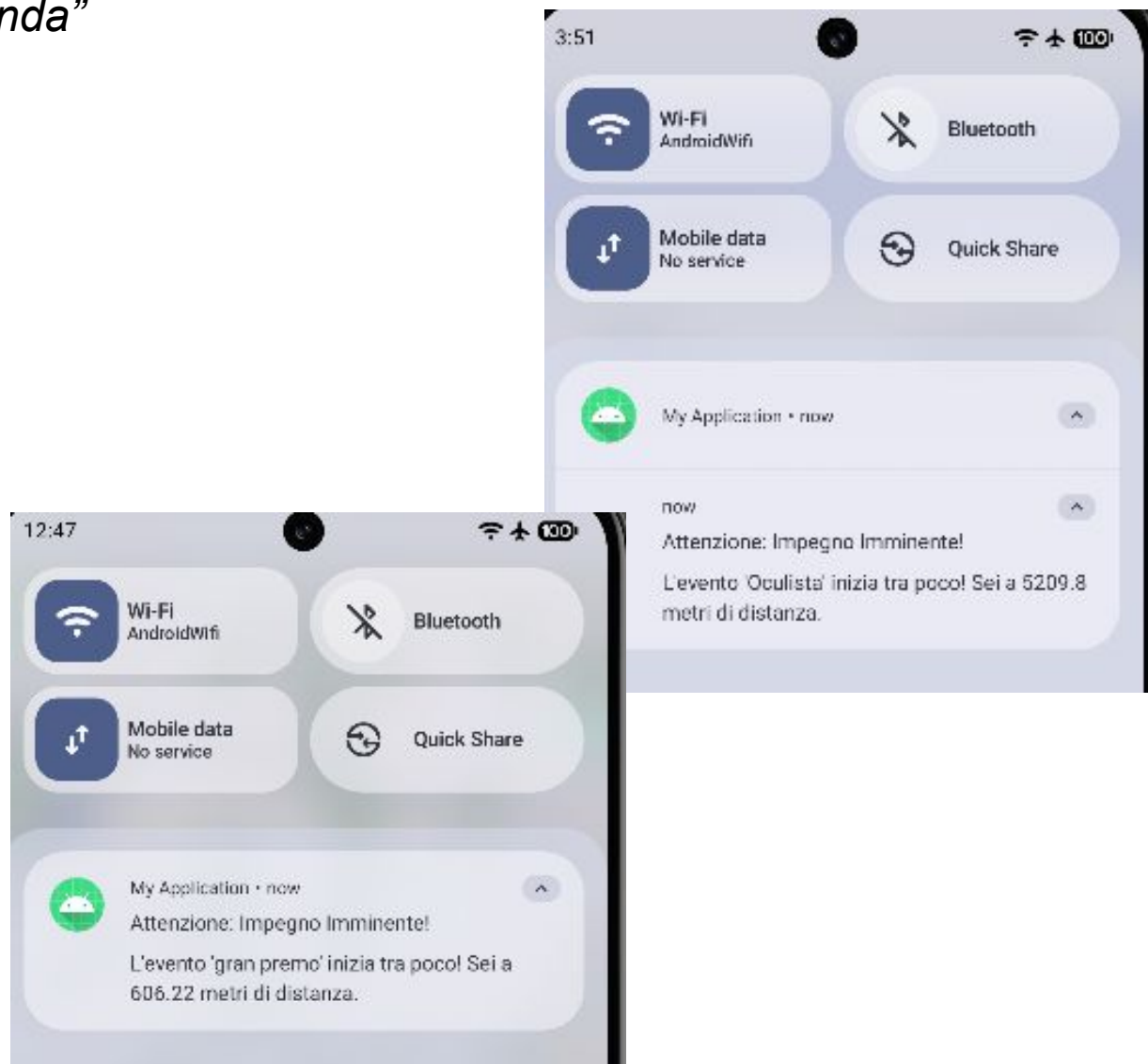
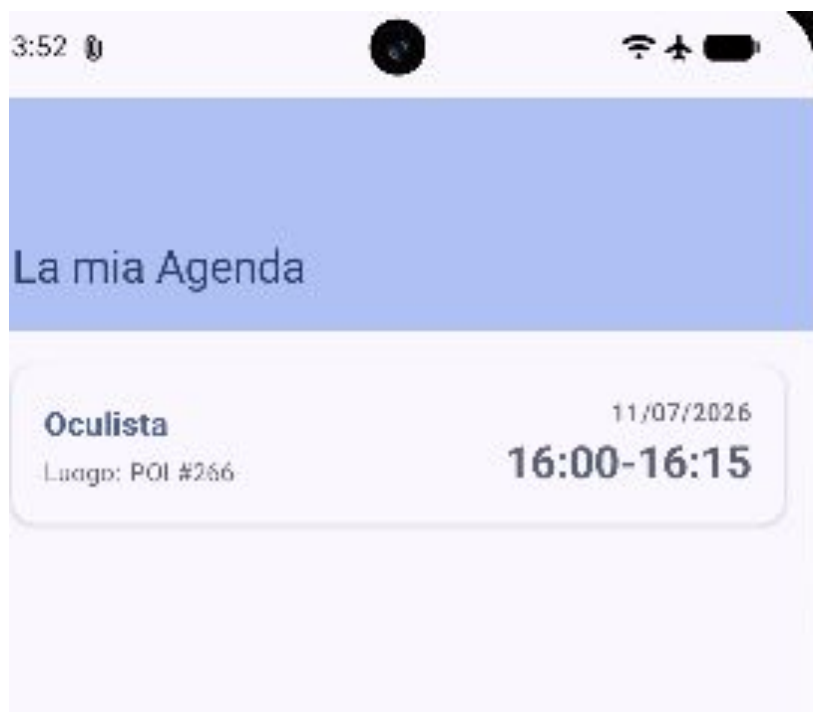
Feedback registrato ✓ ✗

GEOFENCING_ENTER

Mappa Agenda Storico Profilo

App Mobile: Agenda

- L'utente gestisce dall'app una agenda personale dove annotare i suoi impegni e ogni impegno ha:
 - un titolo
 - un intervallo orario
 - il Pol in cui si svolge (con validazione lato server contro le sovrapposizioni)
- Un worker periodico (ogni 15 minuti) interroga la tabella AGENDA_UTENTE per ottenere gli impegni dell'utente in partenza entro 15 minuti
- L'app genera una notifica contestualizzata sia sulla dimensione temporale che su quella spaziale tracciata nello storico come “*Avviso agenda*”





App Mobile: Geofence

- Attorno ai Pol preferiti (`/geofence/config/{id_utente}`) vengono registrate aree circolari tramite le Geofencing API di Google (max 99)
- Quando l'utente attraversa il confine di un'area, un BroadcastReceiver rileva la transizione di
 - entry
 - exit
- Il GeofenceReceiver inoltre:
 - invia la posizione al back-end
 - salva l'evento specificando il motivo
 - mostra una notifica Android nativa

GEOFENCING_EXIT

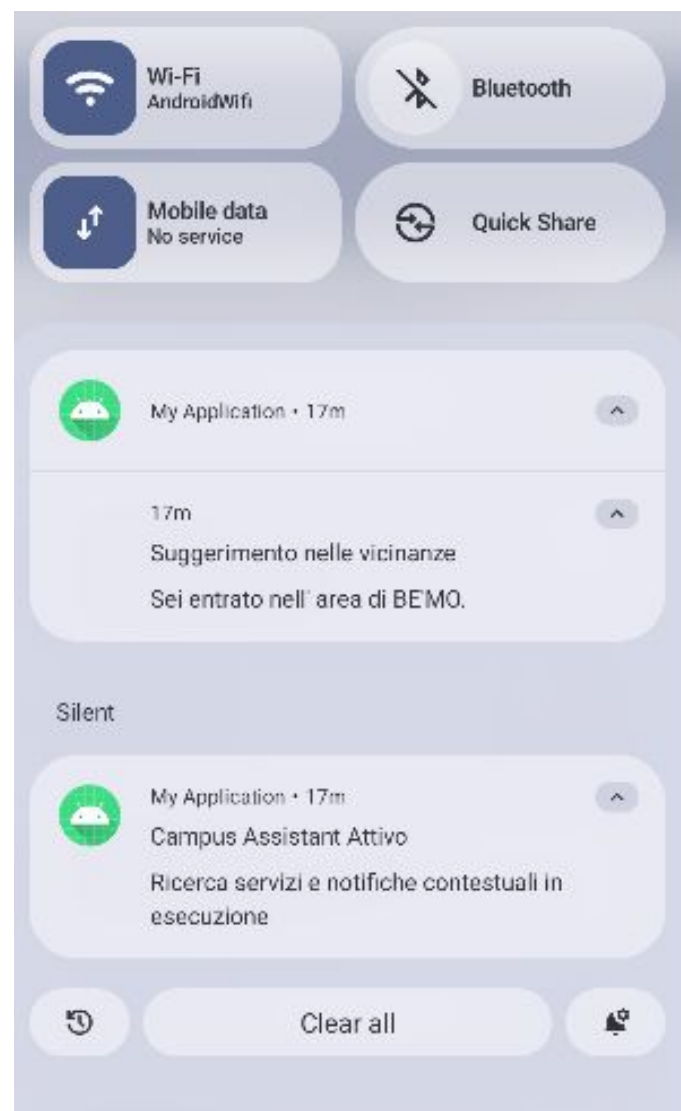
Sei uscito dall' area di Biblioteca Universitaria di Discipline Economico Aziendali "Walter Bigiavi".

Motivo: Geofence scattato - L'utente ha attraversato il confine virtuale del POI: Biblioteca Universitaria di Discipline Economico Aziendali "Walter Bigiavi"

GEOFENCING_ENTER

Sei entrato nell' area di BE'MO.

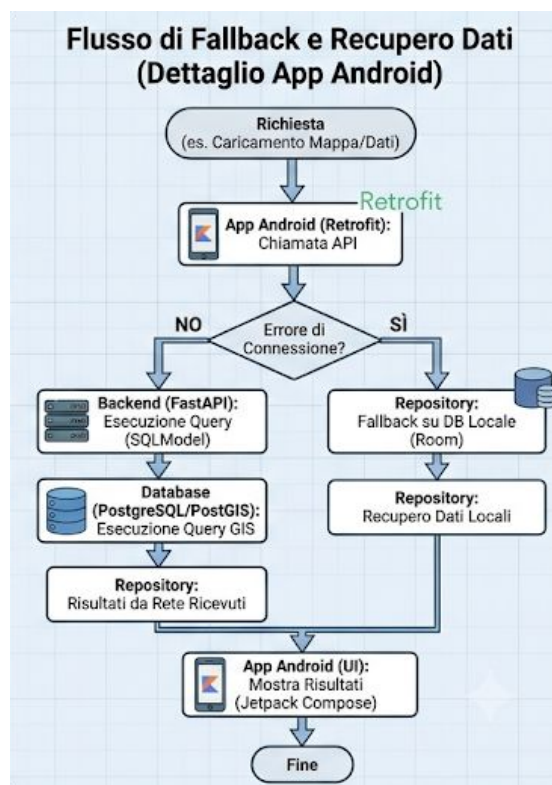
Motivo: Geofence scattato - L'utente ha attraversato il confine virtuale del POI: BE MO



App Mobile: Supporto Offline

L'app mobile riesce ad offrire alcuni servizi anche offline, senza bisogno di una connessione internet grazie ad una cache locale: Room.

- All'avvio, Pol e orari di apertura scaricati dal back-end vengono salvati nel database interno del dispositivo.
- Il Repository adotta una strategia “*remote-first*”:
 - ogni richiesta tenta la chiamata al Backend
 - in caso di errore di connessione, effettua un'operazione sul database di cache
- La ricerca per nome e i filtri per categoria e campus funzionano offline, applicati localmente sui dati salvati
- La sessione utente persiste localmente (SessionManager): dopo un primo accesso l'app è utilizzabile senza ret
- L'agenda può essere consultata anche offline, ma non può essere aggiornata





Dashboard Web (1/2)

- La dashboard permette la gestione CRUD completa dei punti di interesse e il monitoraggio del sistema
- La mappa Leaflet è filtrabile per categoria, campus e orario
- In alto tre indicatori riassumono lo stato:
 - PoI
 - suggerimenti generati
 - percentuale di feedback positivo
- Ogni evento viene diviso per tipologia, orario e filtrabile per data

1088

POI censiti

107

Suggerimenti generati

15%

Feedback positivo

Eventi contestuali per ora del giorno

DAL

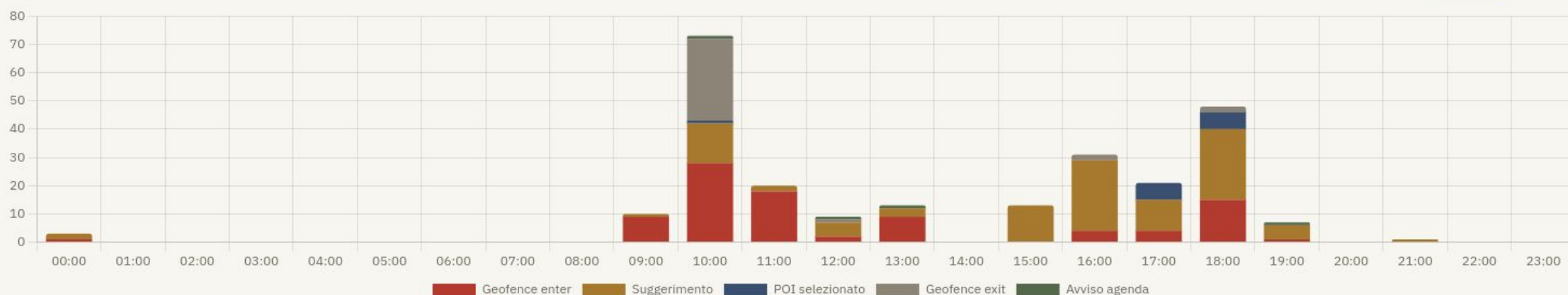
08 / 07 / 2026

AL

15 / 07 / 2026

Applica

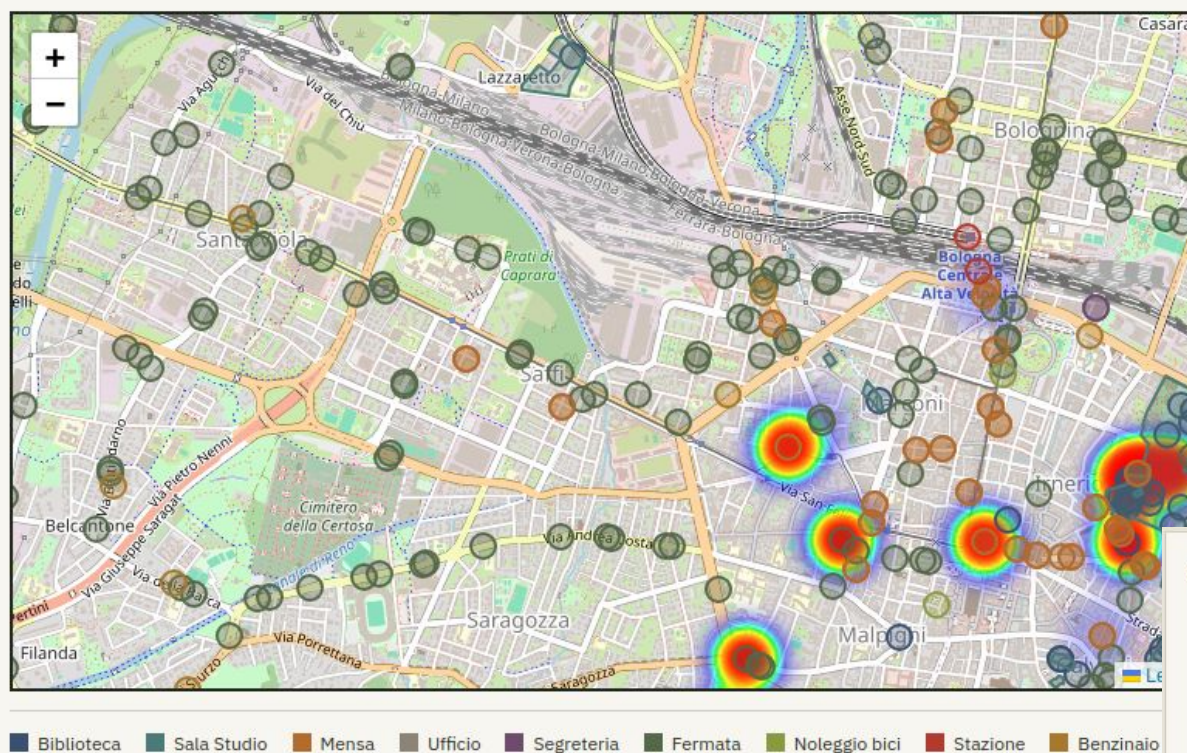
Reset



Dashboard Web (2/2)

- La heatmap evidenzia i punti in cui vengono generati più suggerimenti richiamandoli tramite api (</analytics/heatmap/pois>) ed è attivabile e disattivabile con un pulsante
- Con dei grafici a barre è possibile visualizzare
 - Poi (top 10) con più interazioni, cioè a cui sono stati associati più eventi
 - Classifica Categorie di preferenza, in base agli eventi generati

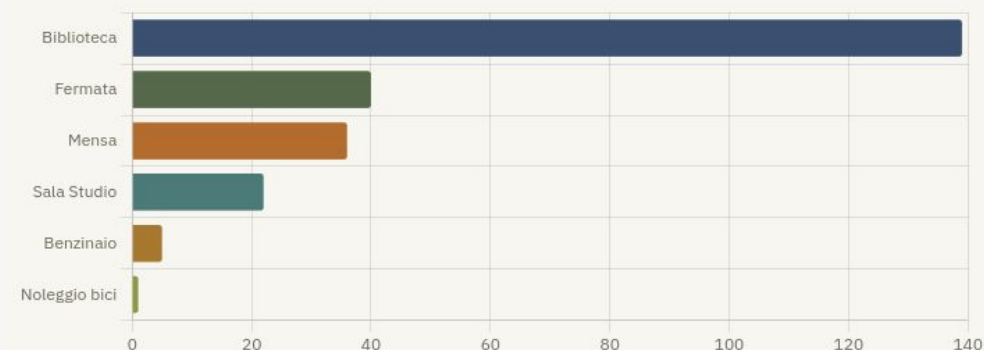
Mappa dei punti di interesse



POI con più interazioni



Categorie più richieste

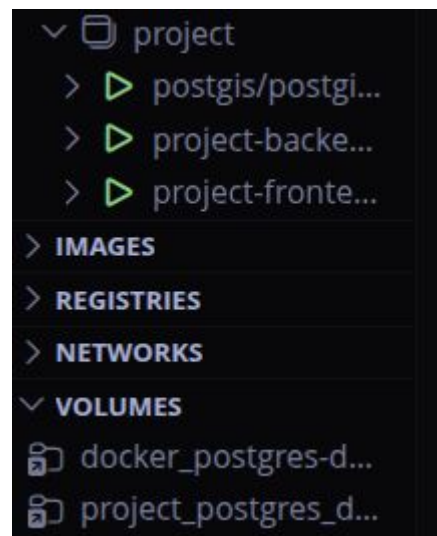




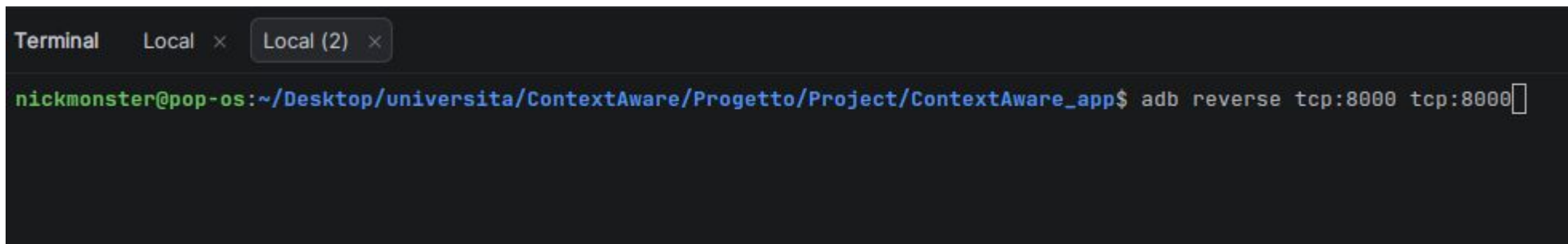
Deployment

L'intera piattaforma server-side si avvia con un solo comando: `docker compose up -d --build`

- **db**, immagine PostGIS, dati su volume per la persistenza
- **backend**, FastAPI su python:3.12-slim, Uvicorn sulla porta 8000
- **frontend**, dashboard statica su Nginx, porta 5500



L'app mobile viene collegata al backend con il comando `adb reverse tcp:8000`





Conclusioni

Il progetto realizza tutti i requisiti dell'applicazione base e sei estensioni della traccia:

- Geo-fencing e notifiche contestuali
- Personalizzazione dei suggerimenti con feedback dell'utente
- Agenda con avvisi contestuali sul prossimo impegno
- Analytics spaziali e temporali
- Supporto offline con caching locale
- Containerizzazione con Docker Compose

Limiti:

- Copertura geofencing: max 99 aree
- Validazione funzionale con singolo utente in ambiente emulato
- Offline in sola consultazione, senza ranking contestuale
- Nessun meccanismo di privacy né autenticazione robusta

Sviluppi futuri:

- tecniche di privacy location aware
- adattamento automatico dei pesi di ranking in base al feedback
- geofence adattabile